Software Architecture 2024[2026]

Lecture 6, part 3/3

Architectural decisions

Dimitri Van Landuyt

Laurens Sion

Wouter Joosen

KU LEUVEN DistriNet

Requirements (ASRs/QASs) (inputs)

Architectural decisions (outputs)

Requirements (ASRs/QASs) (inputs)

Architectural-decision making

Architectural decisions (outputs)

A simple pseudo-algorithm?

```
SA our_architecture = new SA();

for(ASR asr : asrs)

    our_architecture.satisfy(asr);
```

A simple pseudo-algorithm?

```
SA our_architecture = new SA();

for(ASR asr : asrs)

    our_architecture.satisfy(asr);
```

Starting from a black-box perspective?
Starting from an initial design?

A simple pseudo-algorithm?

```
SA our_architecture = new SA();

for(ASR asr : asrs)
    our_architecture.satisfy(asr);
```

Which order? Does it matter?

A simple pseudo-algorithm?

```
SA our_architecture = new SA();

for(ASR asr : asrs)

    our_architecture.satisfy(asr);
```

One by one? Grouping criteria?

A simple pseudo-algorithm?

```
SA our_architecture = new SA();

for(ASR asr : asrs)

    our_architecture.satisfy(asr);
```

Too simplistic, lecture on
Attribute-Driven Design

A simple pseudo-algorithm?

```
SA our_architecture = new SA();

for(ASR asr : asrs)
    our_architecture.satisfy(asr);
```

Today's focus:

How to satisfy one such requirement?

A simple pseudo-algorithm?

```
SA our_architecture = new SA();

for(ASR asr : asrs)
    our_architecture.satisfy(asr);
```

Today's focus:

How to satisfy one such requirement?

"satisfice": satisfy+suffice

Think back

- How did you solve requirements before?
  - Functional vs. non-functional requirements?

Toolbox of a software architect

- Common sense, expertise and experience, creative thinking
- Distilled knowledge (=the "*packaged experience*" of others)
  1. Design principles
  2. Architectural Tactics
  3. Architectural Patterns
  4. Reference architectures
- Methodologies to guide you
  - Pedigreed Attribute Elicitation Method (PALM) (chapter 16)
  - Attribute Driven Design (ADD) (chapter 17)
  - Architecture Trade-Off Analysis Method (ATAM) (chapter 21)

Toolbox of a software architect

1. Common sense, expertise and experience, creative thinking
2. Distilled knowledge (=the "*packaged experience*" of others)
    1. Design principles
    2. Architectural Tactics
    3. Architectural Patterns
    4. Reference architectures

TODAY

3. Methodologies to guide you
    - Pedigreed Attribute Elicitation Method (PALM) (chapter 16)
    - Attribute Driven Design (ADD) (chapter 17)
    - Architecture Trade-Off Analysis Method (ATAM) (chapter 21)

Toolbox of a software architect

1. Common sense, expertise and experience, creative thinking
2. Distilled knowledge (=the "*packaged experience*" of others)
    1. Design principles
    2. Architectural Tactics
    3. Architectural Patterns
    4. Reference architectures
3. Methodologies to guide you

NEXT LECTURE

- ›› Pedigreed Attribute Elicitation Method (PALM) (chapter 16)
- ›› Attribute Driven Design (ADD) (chapter 17)
- ›› Architecture Trade-Off Analysis Method (ATAM) (chapter 21)

1. Design principles

Recap: GRASP

General Responsibility Assignment Software Principles

- Controller: use a non-UI element to deal with system events
- Creator: the creator of an element aggregates it, records it, uses it or has its initialization information
- Indirection: decouple two elements by introducing an intermediary
- Information Expert: responsibility assigned to the element that stores most of the required information
- High Cohesion: give an element strongly related and focused responsibilities
- Low Coupling: avoid dependencies on other elements
- Polymorphism: vary behavior using (sub)types
- Protected Variations: use interfaces and polymorphism to hide (encapsulate) variations
- Pure Fabrication: non-domain element with sole purpose of achieving low coupling/high cohesion

Recap: GRASP

General Responsibility Assignment Software Principles

- Controller: use a non-UI element to deal with system events
- Creator: the creator of an element aggregates it, records it, uses it or has its initialization information
- Indirection: decouple two elements by introducing an intermediary
- Information Expert: responsibility assigned to the element that stores most of the required information
- High Cohesion: give an element strongly related and focused responsibilities
- Low Coupling: avoid dependencies on other elements
- Polymorphism: vary behavior using (sub)types
- Protected Variations: use interfaces and polymorphism to hide (encapsulate) variations
- Pure Fabrication: non-domain element with sole purpose of achieving low coupling/high cohesion

Exercise: Which qualities are impacted by these principles?

SOLID principles

- Single responsibility principle
  - An element should have a single responsibility (reason to change)
- Open/closed principle
  - Open for extension, closed for modification
- Liskov substitution principle
  - Subtypes can always replace supertypes
- Interface segregation principle
  - Avoid large general purpose interfaces
- Dependency inversion principle
  - Depend on abstractions, not on details

SOLID principles

- Single responsibility principle
  - An element should have a single responsibility (reason to change)
- Open/closed principle
  - Open for extension, closed for modification
- Liskov substitution principle
  - Subtypes can always replace supertypes
- Interface segregation principle
  - Avoid large general purpose interfaces
- Dependency inversion principle
  - Depend on abstractions, not on details

Exercise: Which qualities are impacted by these principles?

Some more design principles

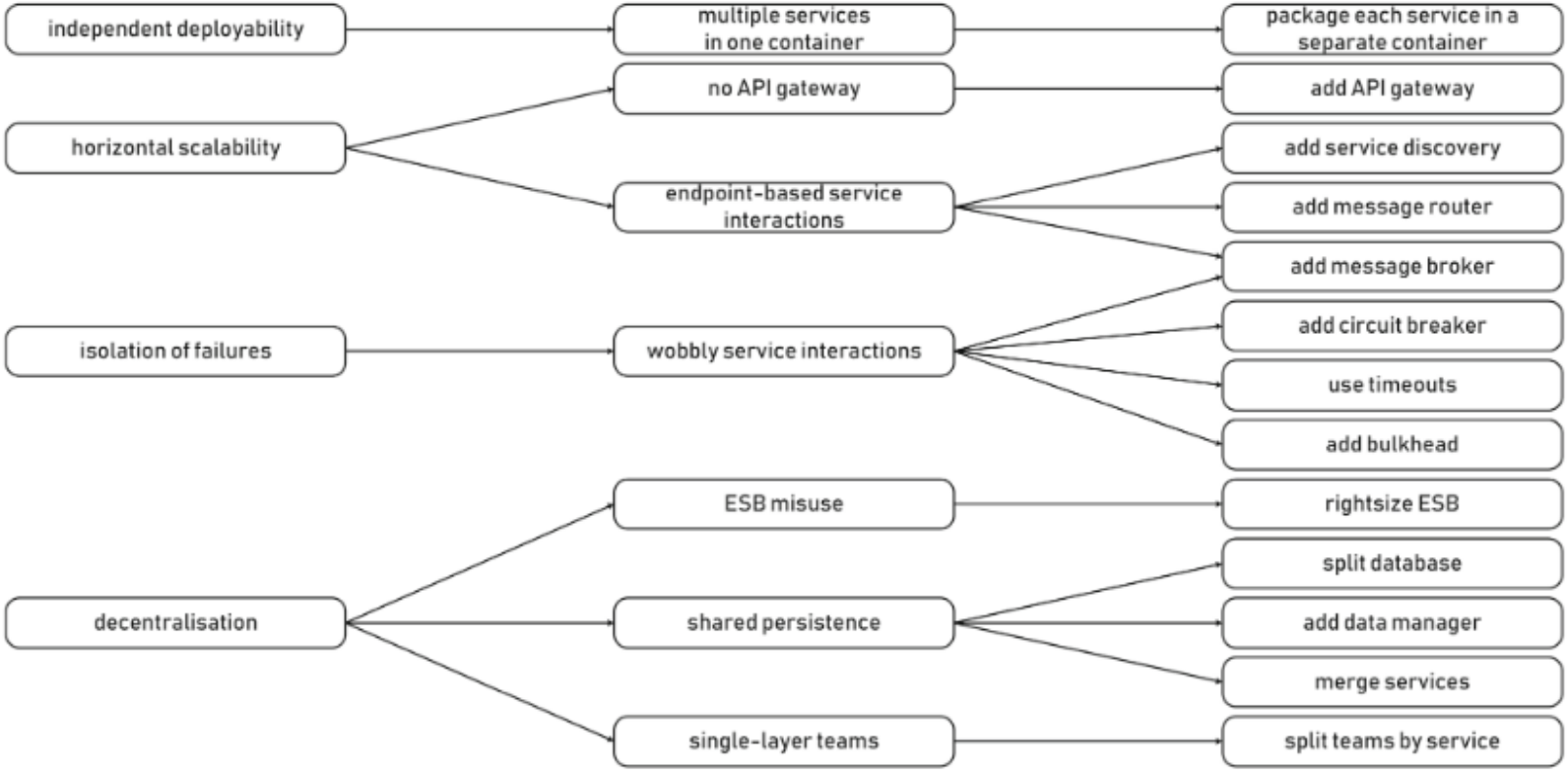
- Keep components small by giving each component a single, clearly defined purpose
- Give components simple, understandable interfaces
- Keep frequently interacting components close
- Distribute data sources (and keep them close to consumers)
- Minimize dependencies and remove unnecessary ones (keep coupling low)
- Maximize cohesiveness of each component
- Use hierarchical (de)composition
- Avoid single points of failure
- Avoid hard-coded limits

Some more design principles

- Keep components small by giving each component a single, clearly defined purpose
- Give components simple, understandable interfaces
- Keep frequently interacting components close
- Distribute data sources (and keep them close to consumers)
- Minimize dependencies and remove unnecessary ones (keep coupling low)
- Maximize cohesiveness of each component
- Use hierarchical (de)composition
- Avoid single points of failure
- Avoid hard-coded limits

Exercise: Which qualities are impacted by these principles?

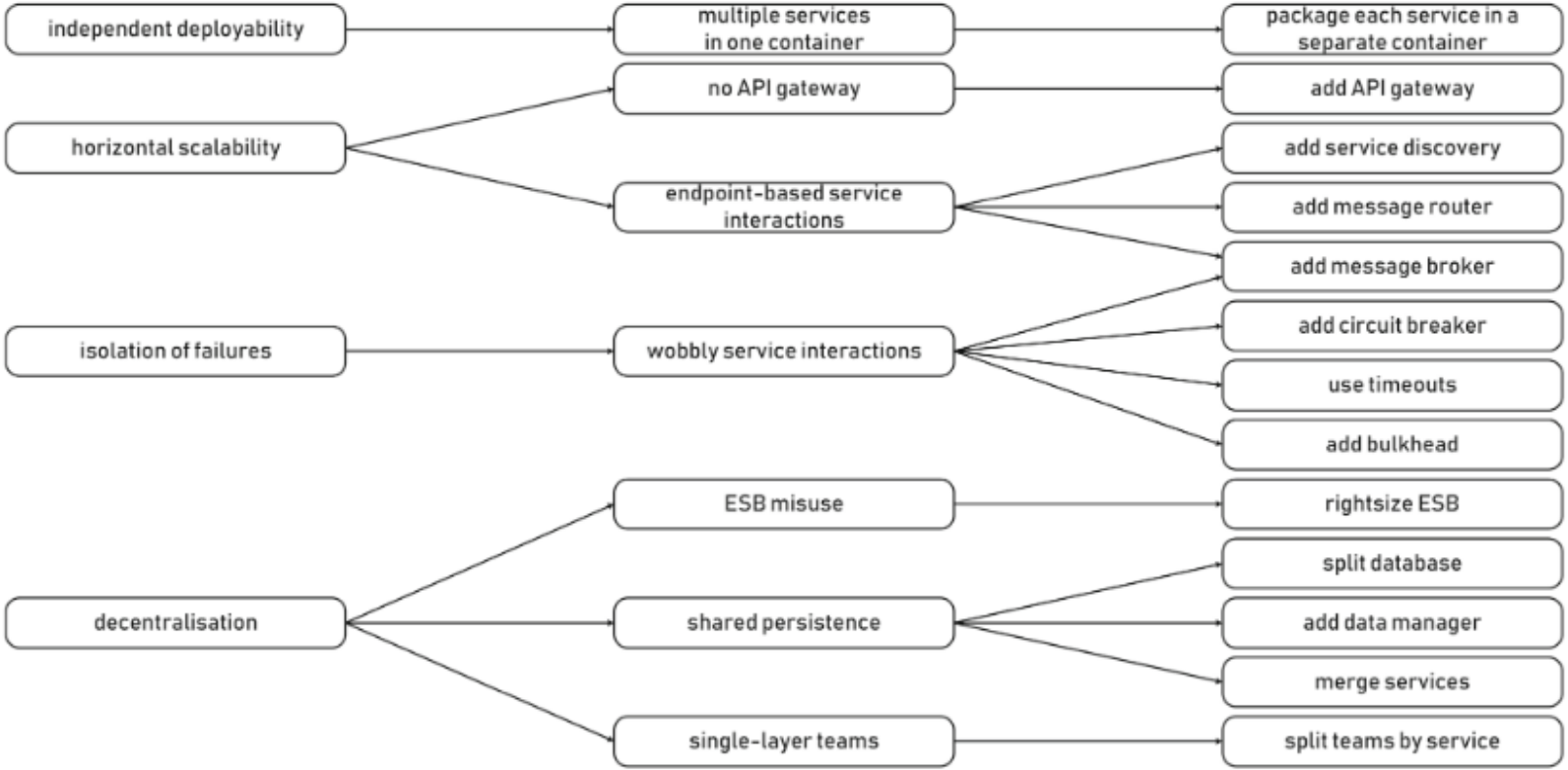
Design principles for microservice architectures



Neri, Davide, et al. "Design principles, architectural smells and refactorings for microservices: a multivocal review." SICS Software-Intensive Cyber-Physical Systems (2019): 1-13.

KU LEUVEN DistriNet

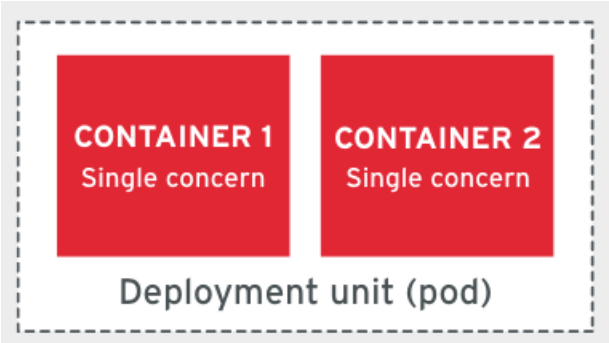
Design principles for microservice architectures



Exercise: Which qualities are impacted by these principles?

Container-based application design

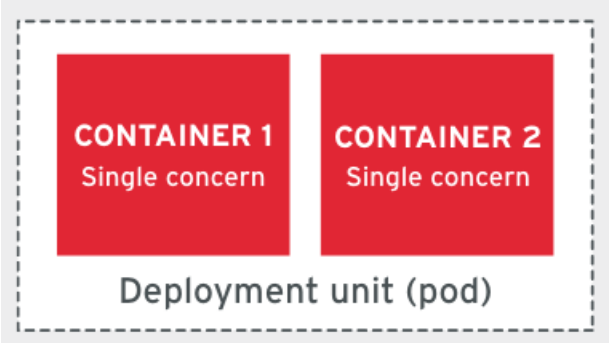
- Single Concern Principle (SCP)
- High Observability Principle (HOP)
- Life-cycle Conformance Principle (LCP)
- Image Immutability Principle (IIP)
- Process Disposability Principle (PDP)
- Self-Containment Principle (S-CP)
- Runtime Confinement Principle (RCP)



KU LEUVEN DistriNet

Container-based application design

- Single Concern Principle (SCP)
- High Observability Principle (HOP)
- Life-cycle Conformance Principle (LCP)
- Image Immutability Principle (IIP)
- Process Disposability Principle (PDP)
- Self-Containment Principle (S-CP)
- Runtime Confinement Principle (RCP)



Exercise: Which qualities are impacted by these principles?

Microsoft Azure Well-architected framework

Security design principles

- Plan resources and how to harden them
- Automate and use least privilege
- Classify and encrypt data
- Monitor system security, plan incident response
- Identify and protect endpoints
- Protect against code-level vulnerabilities
- Model and test against potential threats

https://learn.microsoft.com/en-us/azure/architecture/framework/security/security-principles

Security By Design Principles (OWASP)

1. Minimise attack surface area
2. Establish secure defaults
3. The principle of Least privilege
4. The principle of Defence in depth
5. Fail securely
6. Don’t trust services
7. Separation of duties
8. Avoid security by obscurity
9. Keep security simple
10. Fix security issues correctly

https://owasp.org/www-project-security-knowledge-framework/

You can study design principles *all day*

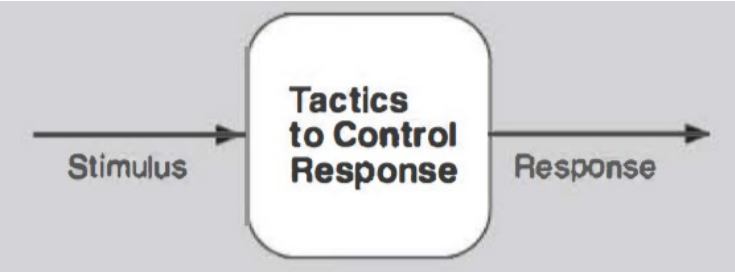
- Design principles per programming paradigm (e.g., OO)
- Design principles per technology stack
- Design principles per non-functional requirement (e.g. security)
- Design principles per application domain (e.g. e-health)
- Or, the reverse: bad smells/anti-patterns/...

Plenty of resources: Books, online references, articles, whitepapers

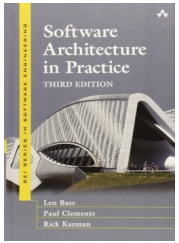
2. Architectural tactics

Tactics

- A tactic is a common design decision that influences the achievement of a quality attribute response
  - Represents a **common solution** to a common requirement
  - Focused on a **single attribute**: no trade-offs made within the tactic
  - Describes a decision in terms of the system's response to a stimulus

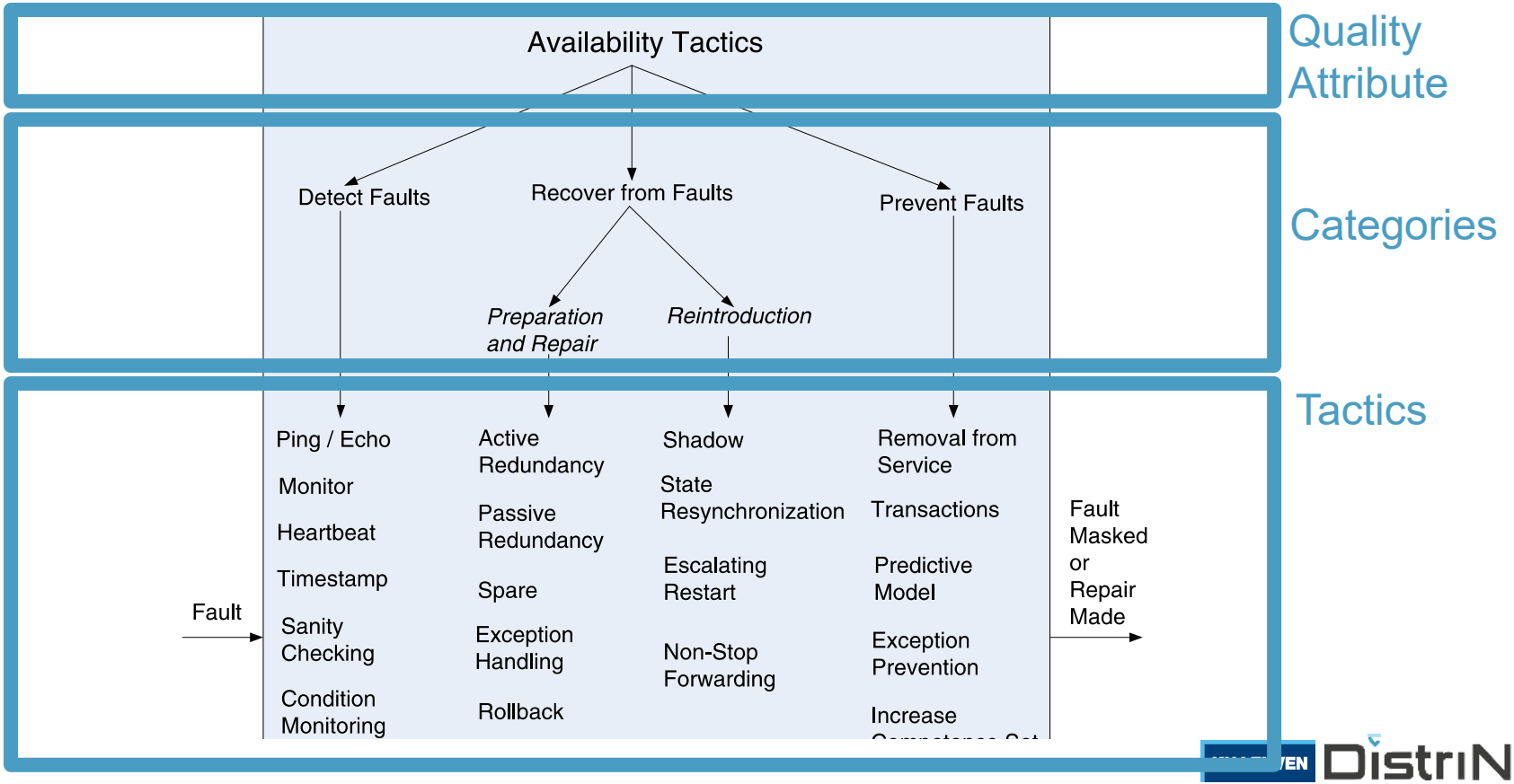


- "*To meet X-ility, make your system do Y*"
- Not every tactic suits every system!



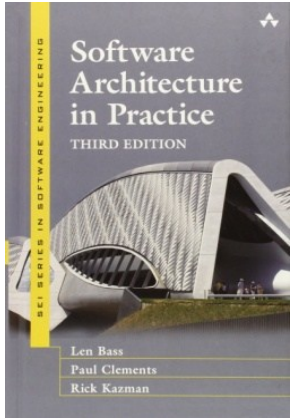
Section 4.5

Structure of a tactic tree



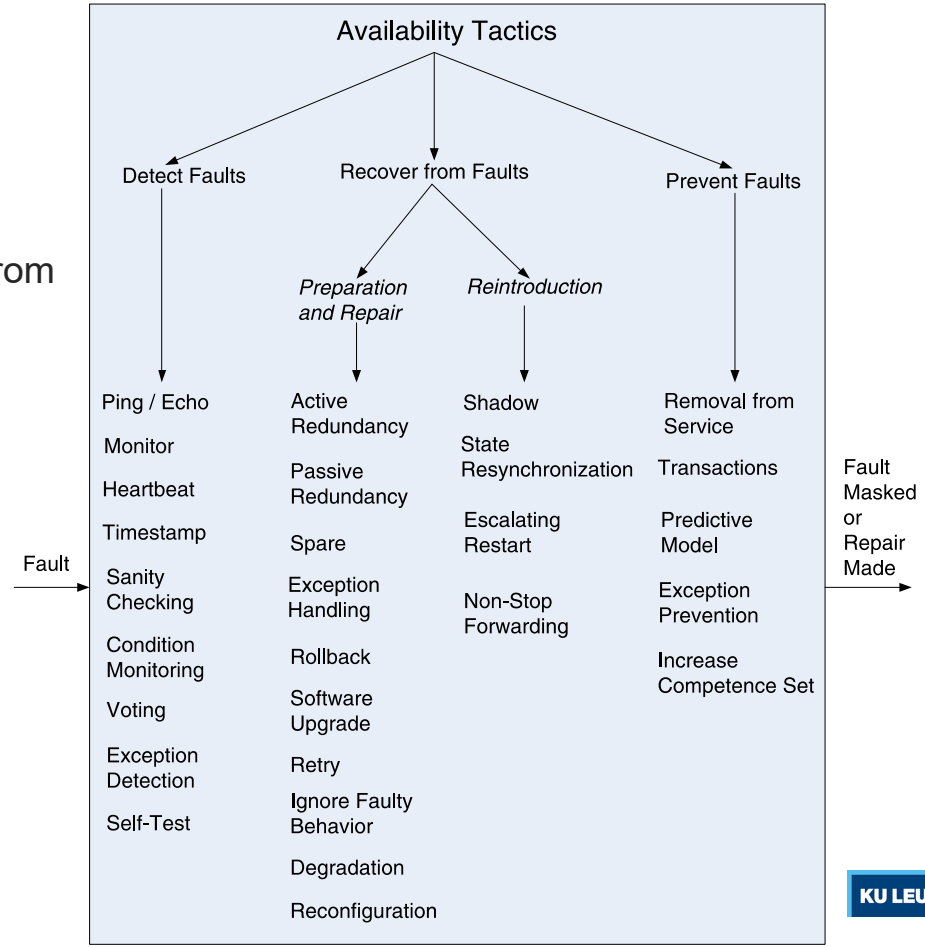
Tactics for

- Availability – chapter 5
- Interoperability – chapter 6
- Modifiability – chapter 7
- Performance – chapter 8
- Security – chapter 9
- Usability – chapter 10
- Testability – chapter 11



Availability tactics (SAiP Ch. 5)

Fault (something that happens, possibly causing a failure)
vs.
Failure (system visibly deviates from specification)



Detect Faults

Reference

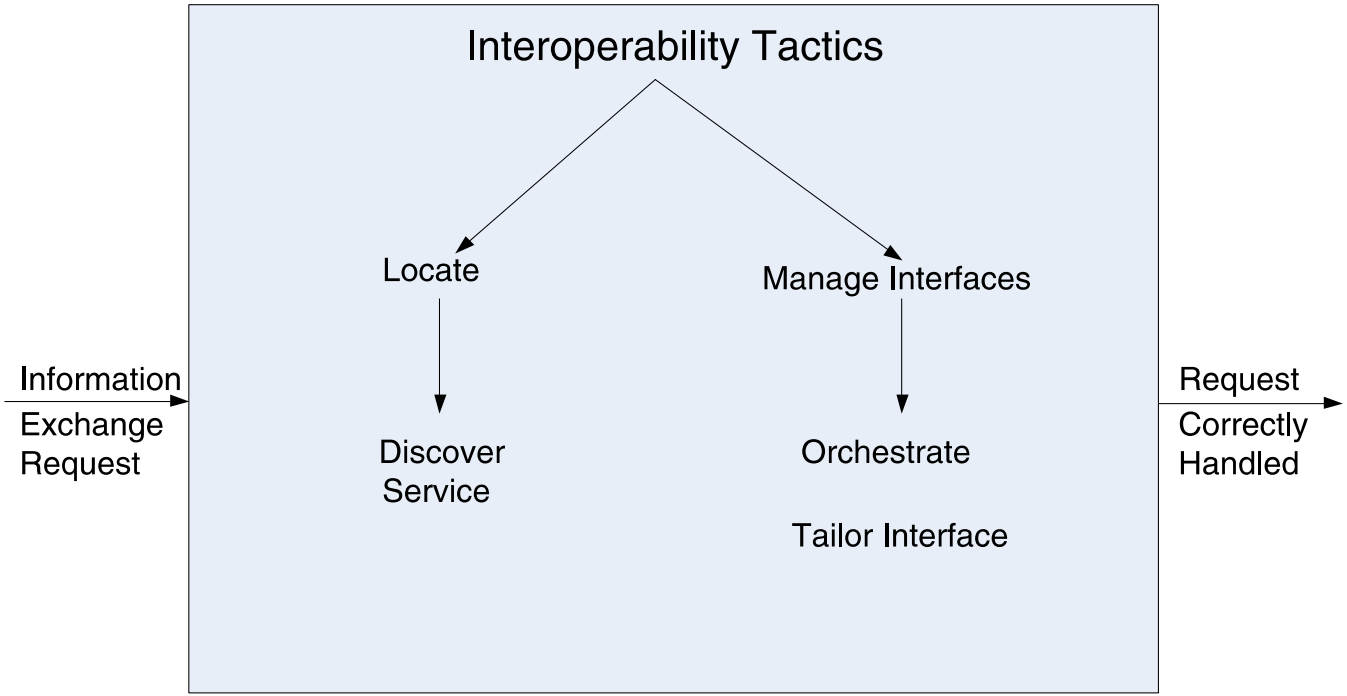
- **Ping/echo**: asynchronous request/response message pair exchanged between nodes, used to determine reachability and the round-trip delay through the associated network path.
- **Monitor**: a component used to monitor the state of health of other parts of the system. A system monitor can detect failure or congestion in the network or other shared resources, such as from a denial-of-service attack.
- **Heartbeat**: a periodic message exchange between a system monitor and a process being monitored.

Summaries for other tactics

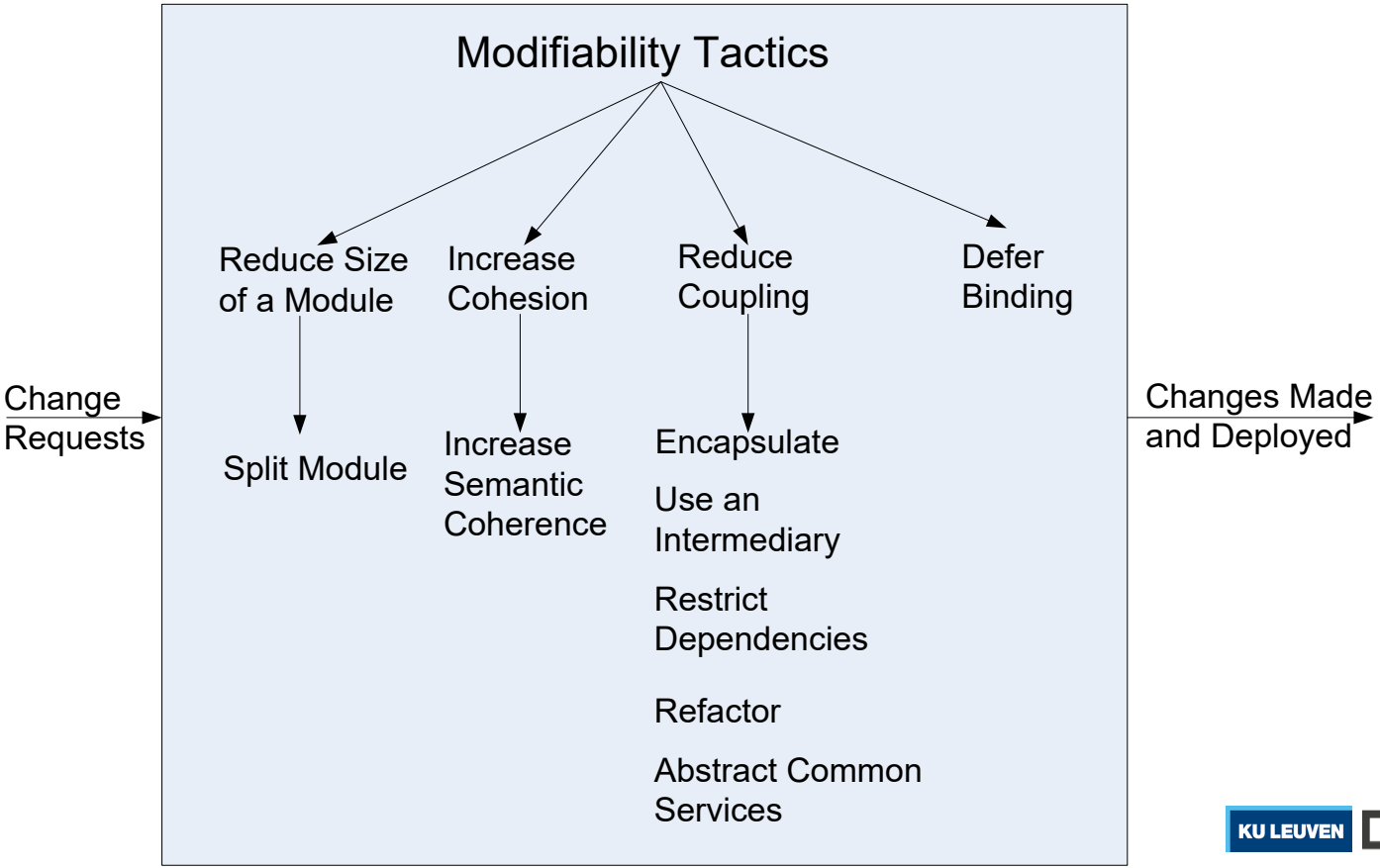
Reference

- See separate ‘Tactics reference’ slides on Toledo

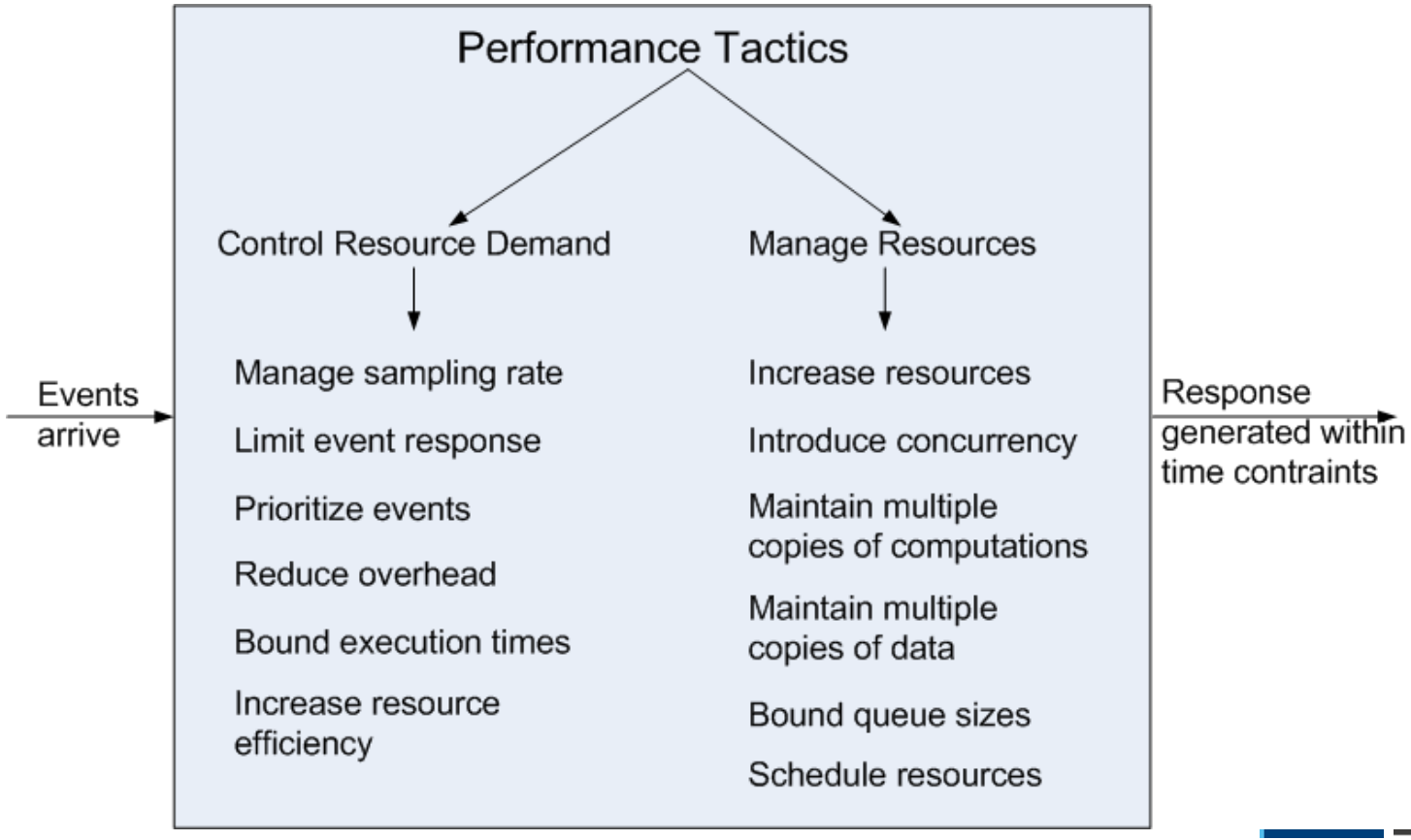
Interoperability tactics (SAiP Ch. 6)



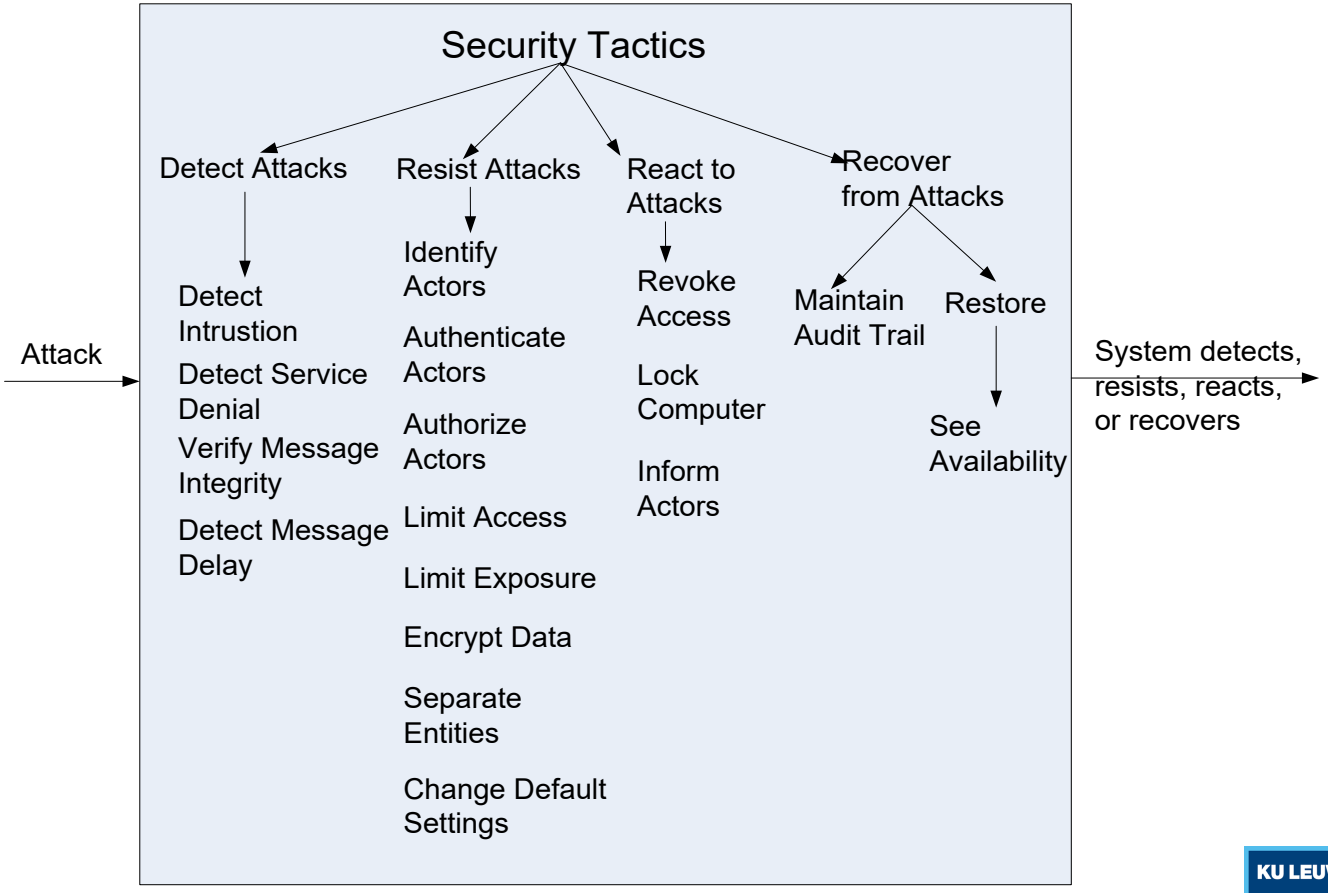
Modifiability tactics (SAiP Ch. 7)



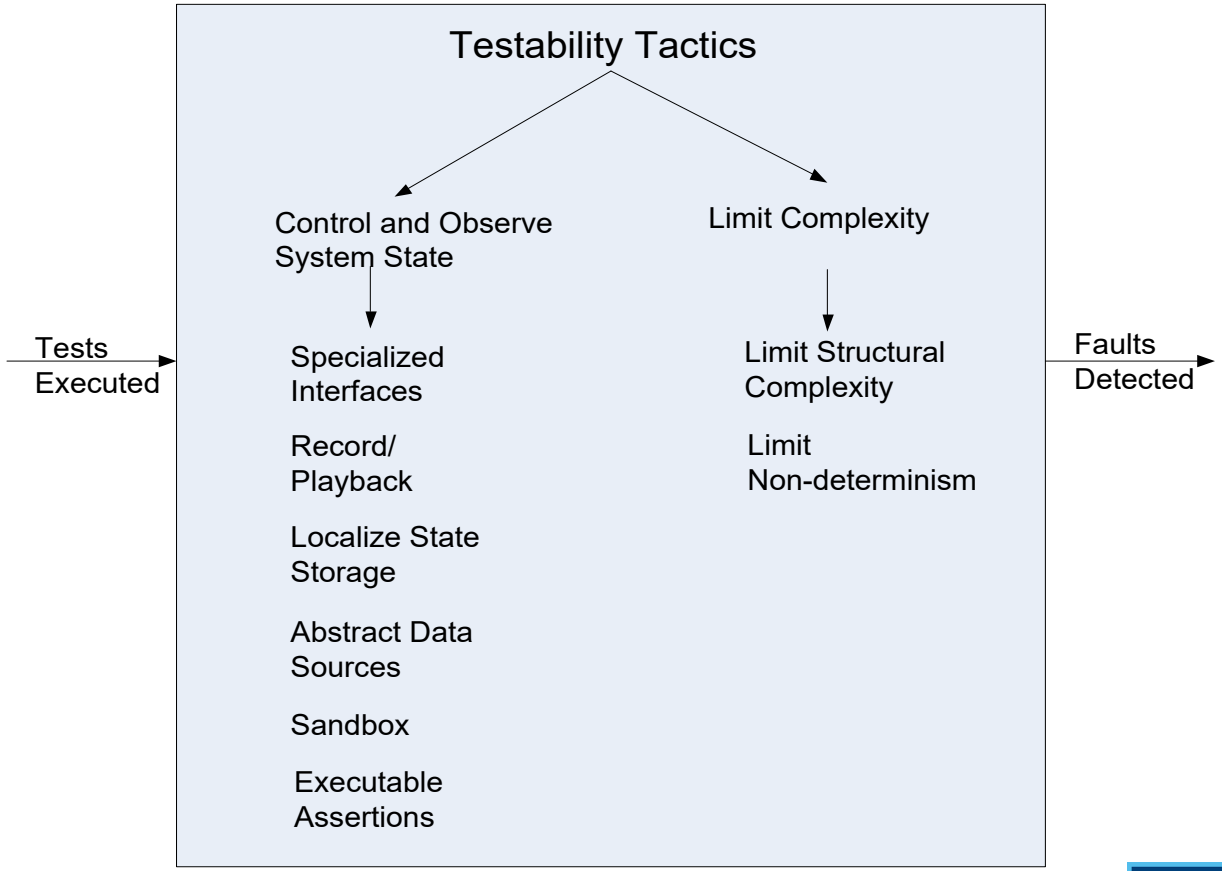
Performance tactics (SAiP Ch. 8)



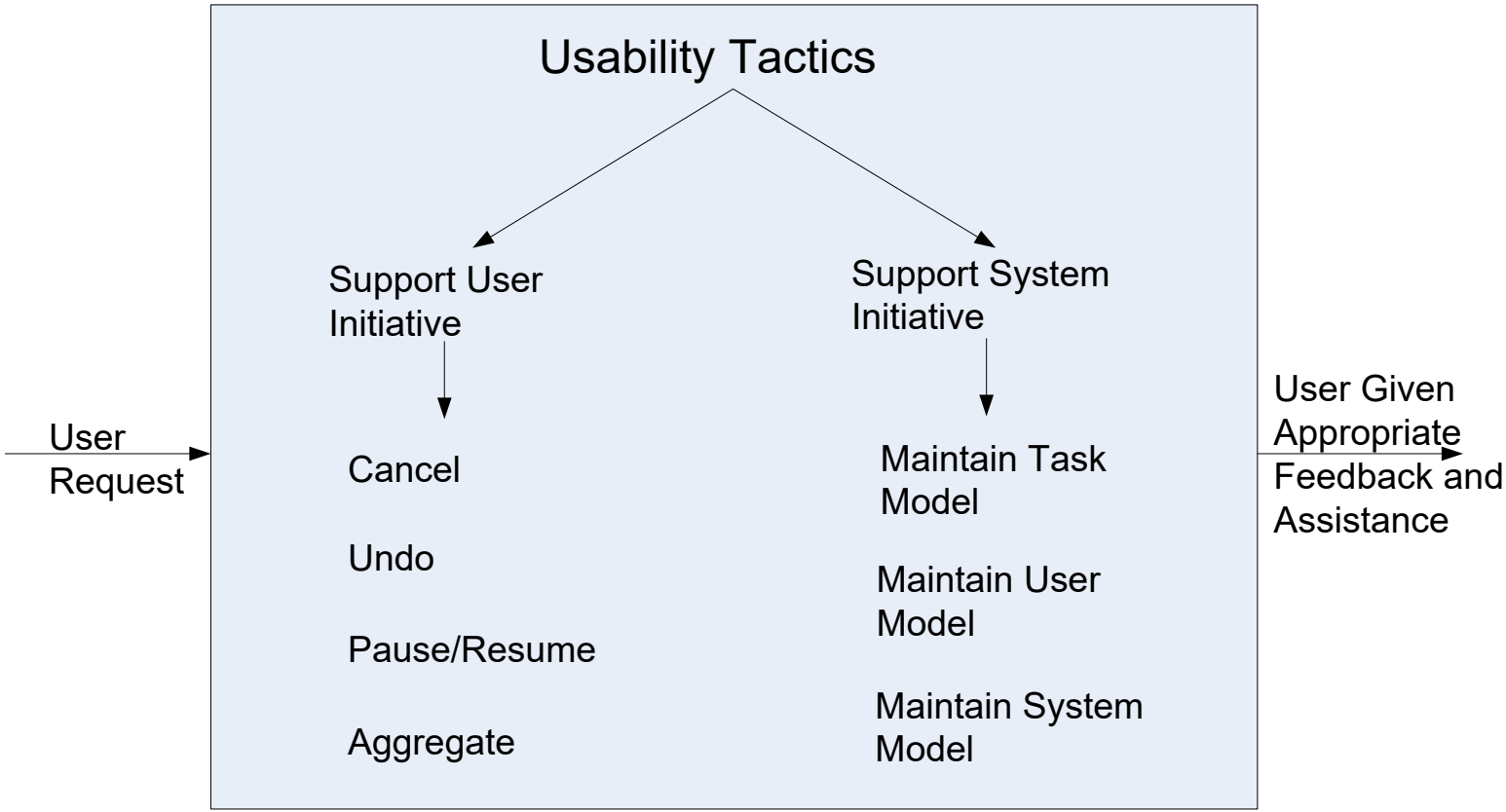
Security tactics (SAiP Ch. 9)



Testability tactics (SAiP Ch. 10)

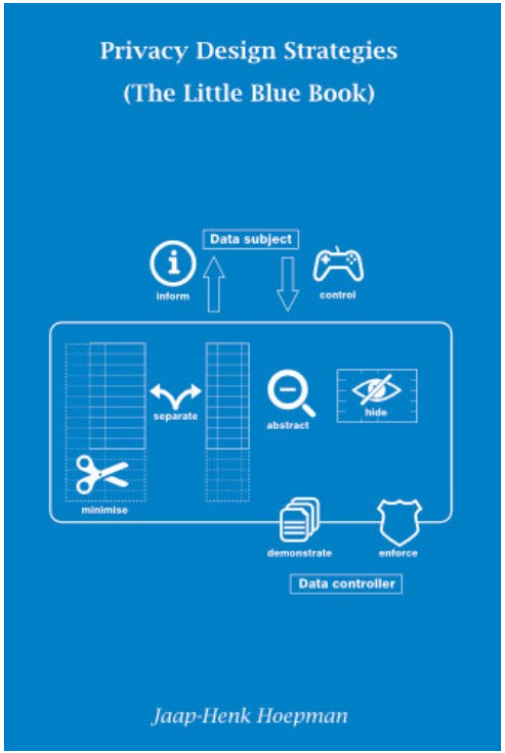


Usability tactics (SAiP Ch. 11)



Privacy tactics/strategies

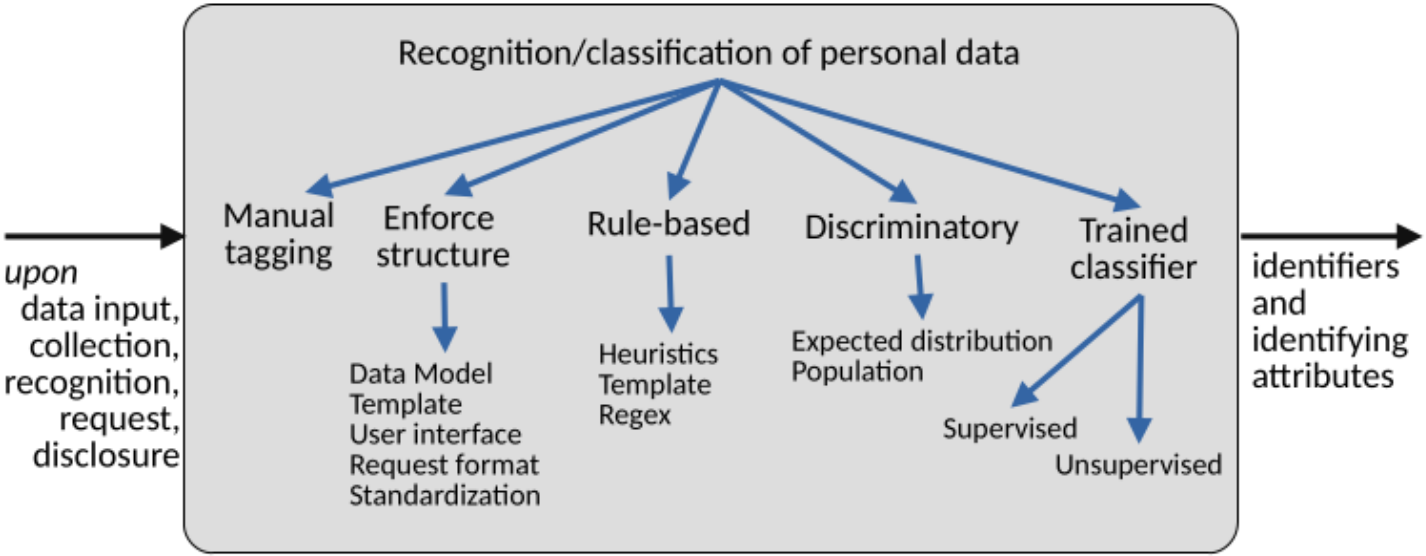
MINIMISE	HIDE	SEPARATE	ABSTRACT
EXCLUDE SELECT STRIP DESTROY	RESTRICT MIX OBFUSCATE DISSOCIATE	DISTRIBUTE ISOLATE	SUMMARIZE GROUP
INFORM	**CONTROL**	**ENFORCE**	**DEMONSTRATE**
SUPPLY NOTIFY EXPLAIN	CONSENT CHOOSE UPDATE RETRACT	CREATE MAINTAIN UPHOLD	AUDIT LOG REPORT



M. Colesky, J. -H. Hoepman and C. Hillen, "A Critical Analysis of Privacy Design Strategies," 2016 IEEE Security and Privacy Workshops (SPW), San Jose, CA, USA, 2016, pp. 33-40

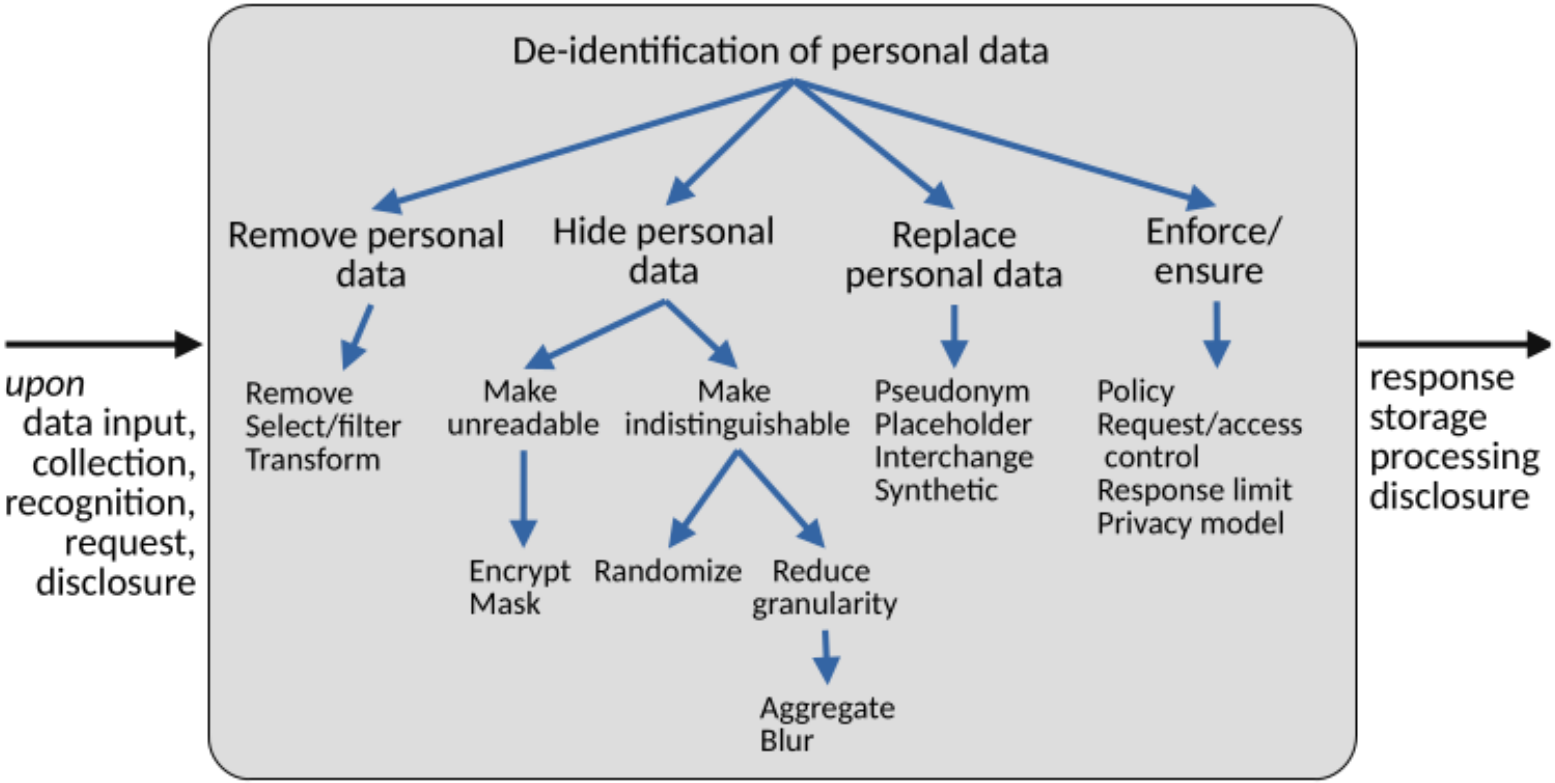
https://www.cs.ru.nl/~jhh/blue-book.html

Tactics for de-identification (privacy)



Dimitri Van Landuyt, Wouter Joosen, A Systematic Survey of Architectural Approaches and Trade-Offs in Data De-identification, 16th European Conference on Software Architecture (ECSA), Prague, Czech Republic, September 19-23, 2022

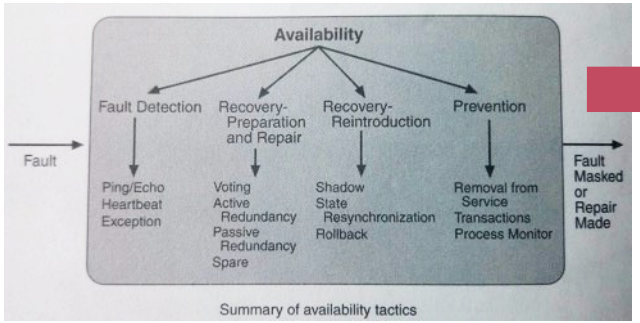
Tactics for de-identification (privacy)



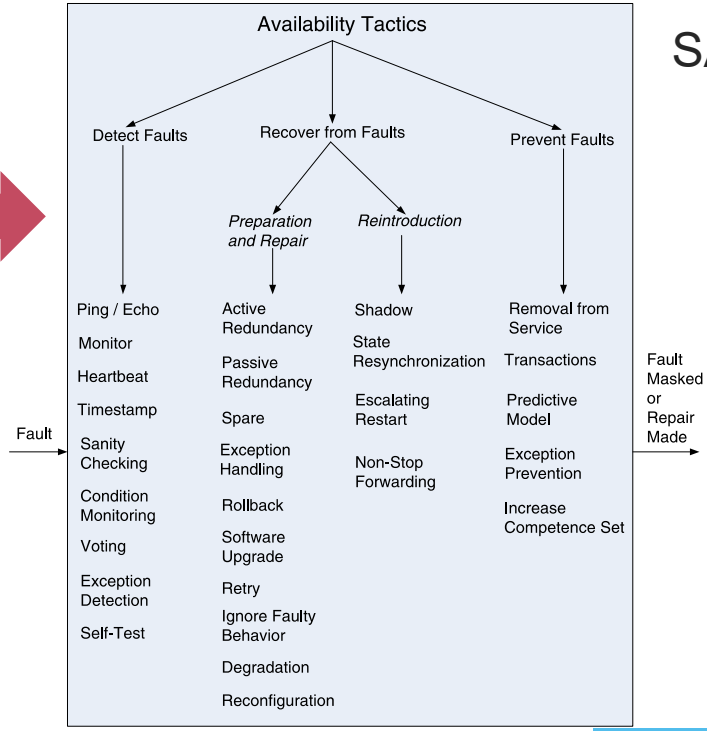
Dimitri Van Landuyt, Wouter Joosen, A Systematic Survey of Architectural Approaches and Trade-Offs in Data De-identification, 16th European Conference on Software Architecture (ECSA), Prague, Czech Republic, September 19-23, 2022

Note about tactics

Tactics are not defined "*once and for all*"!



SAiP 2nd edition



SAiP 3rd edition

3. Architectural Patterns

Patterns for solving problems

“A pattern describes

a problem which occurs over and over again [..]

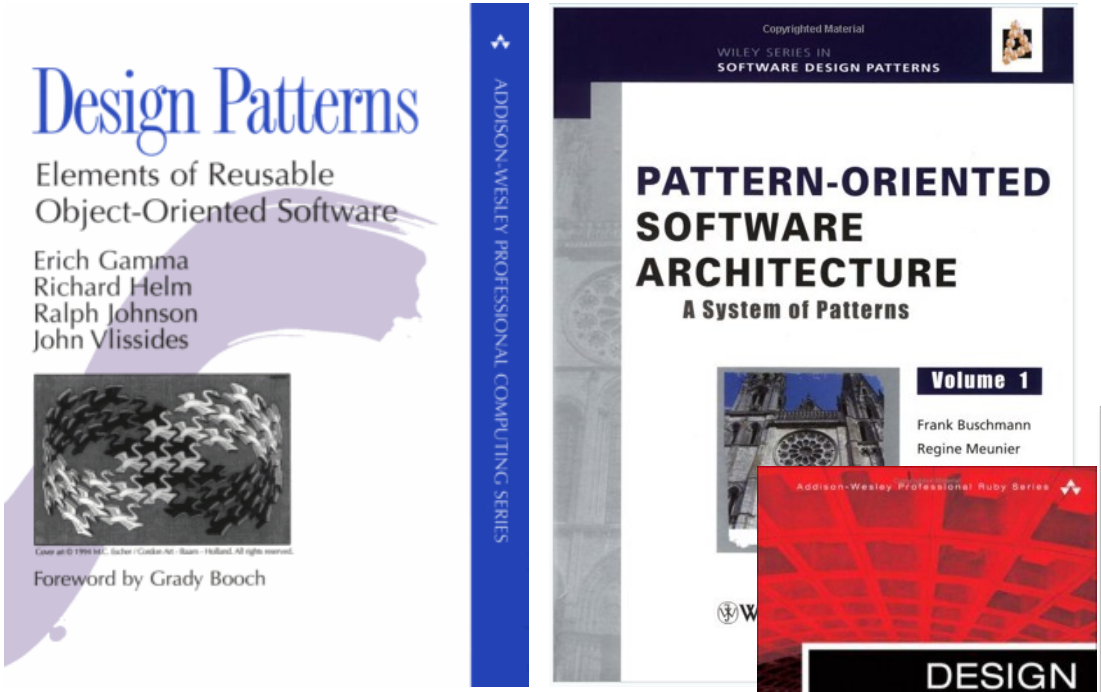
and then describes the core of the solution to that problem

in such a way that you can use this solution a million times over,

without ever doing it the same way twice.”

— Christopher Alexander, A Pattern Language, 1977

Patterns for software



Architectural patterns

- A reusable “piece” of architectural design
  - Not just a single decision, but a fragment/structure in which one or more decisions have been made
  - Needs to be instantiated – generic roles have to be mapped to concrete system elements

Architectural patterns

- Named collection of architectural fragments
  - Applicable to a recurring design problem
  - Parameterized and comes in variants to account for different contexts
  - Known trade-offs, consequences, pitfalls
- Blurry distinction between architectural patterns and architectural styles
  - Styles are broader and more generally applicable
  - In this course, we consider them the same
- **Patterns** package one or more **tactics**
  - Trade-offs are built in
  - Distinction can be blurry as well, though

Example: modifiability tactics & patterns

Table 1: Architectural Patterns and Corresponding Tactics

Pattern	Modifiability									
	Increase Cohesion		Reduce coupling					Defer Binding Time		
	Maintain Semantic Coherence	Abstract Common Services	Use Encapsulation	Use a Wrapper	Restrict Comm. Paths	Use an Intermediary	Raise the Abstraction Level	Use Runtime Registration	Use Start-Up Time Binding	Use Runtime Binding
Layers	X	X	X		X	X	X			
Pipe-and-Filter	X		X		X	X			X	
Blackboard	X	X			X	X	X	X		X
Broker	X	X	X		X	X	X	X		
Model-View-Controller	X		X			X				X
Presentation-Abstraction-Control	X		X			X	X			
Microkernel	X	X	X		X	X				
Reflection	X		X							

Modifiability Tactics (2007)
https://resources.sei.cmu.edu/asset_files/TechnicalReport/2007_005_001_14858.pdf

Patterns in SAiP3 (Chapter 13)

- **Module patterns**
  - Layers
- **Component and connector patterns**
  - Broker
  - Model-View-Controller
  - Pipe and Filter
  - Client-Server
  - Peer-to-Peer
  - Service-Oriented Architecture
  - Publish-Subscribe
  - Shared Data
- **Allocation patterns**
  - Map-Reduce
  - Multi-Tier

Patterns in POSA Vol. 4

From Mud To Structure: DOMAIN MODEL (182), LAYERS (185), MODEL-VIEW-CONTROLLER (188), PRESENTATION-ABSTRACTION-CONTROL (191), MICROKERNEL (194), REFLECTION (197), PIPES AND FILTERS (200), SHARED REPOSITORY (202), BLACKBOARD (205), and DOMAIN OBJECT (208).

Distribution Infrastructure: MESSAGING (221), MESSAGE CHANNEL (224), MESSAGE ENDPOINT (227), MESSAGE TRANSLATOR (229), MESSAGE ROUTER (231), BROKER (237), CLIENT PROXY (240), REQUESTOR (242), INVOKER (244), CLIENT REQUEST HANDLER (246), SERVER REQUEST HANDLER (249), and PUBLISHER-SUBSCRIBER (234).

Event Demultiplexing and Dispatching: REACTOR (259), PROACTOR (262), ACCEPTOR-CONNECTOR (265), and ASYNCHRONOUS COMPLETION TOKEN (268).

Interface Partitioning: EXPLICIT INTERFACE (281), EXTENSION INTERFACE (284), INTROSPECTIVE INTERFACE (286), DYNAMIC INVOCATION INTERFACE (288), PROXY (290), BUSINESS DELEGATE (292), FACADE (294), COMBINED METHOD (296), ITERATOR (298), ENUMERATION METHOD (300), and BATCH METHOD (302).

Component Partitioning: ENCAPSULATED IMPLEMENTATION (313), WHOLE-PART (317), COMPOSITE (319), MASTER-SLAVE (321), HALF-OBJECT PLUS PROTOCOL (324), and REPLICATED COMPONENT GROUP (326).

Application Control: PAGE CONTROLLER (337), FRONT CONTROLLER (339), APPLICATION CONTROLLER (341), COMMAND PROCESSOR (343), TEMPLATE VIEW (345), TRANSFORM VIEW (347), FIREWALL PROXY (349), and AUTHORIZATION (351).

Concurrency: HALF-SYNC/HALF-ASYNC (359), LEADER/FOLLOWERS (362), ACTIVE OBJECT (365), MONITOR OBJECT (368).

Synchronization: GUARDED SUSPENSION (380), FUTURE (382), THREAD-SAFE INTERFACE (384), DOUBLE- CHECKED LOCKING (386), STRATEGIZED LOCKING (388), SCOPED LOCKING (390), THREAD-SPECIFIC STORAGE (392), COPIED VALUE (394), and IMMUTABLE VALUE (396).

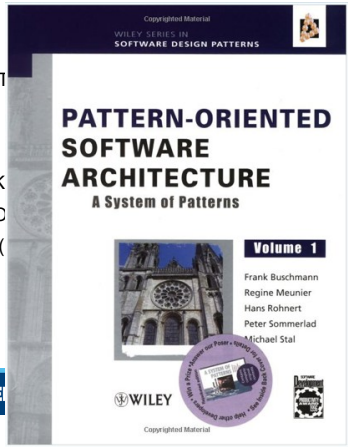
Object Interaction: OBSERVER (405), DOUBLE DISPATCH (408), MEDIATOR (410), MEMENTO (414), CONTEXT OBJECT (416), DATA TRANSFER OBJECT (418), COMMAND

Adaptation and Extension: BRIDGE (436), OBJECT ADAPTER (438), INTERCEPTOR (444), CHAIN OF RESPONSIBILITY (440), INTERPRETER (442), VISITOR (447), DECORAT (453), STRATEGY (455), NULL OBJECT (457), WRAPPER FACADE (459), EXECUTE-AROUND OBJECT (451), and DECLARATIVE COMPONENT CONFIGURATION (461).

Object Behavior: OBJECTS FOR STATES (467), METHODS FOR STATES (469), and COLLECTIONS FOR STATES (471).

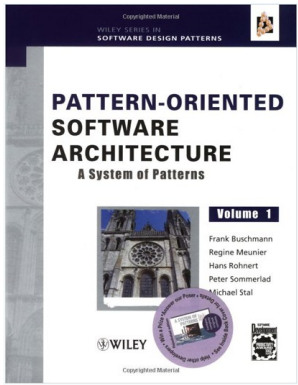
Resource Management: OBJECT MANAGER (492), CONTAINER (488), COMPONENT CONFIGURATOR (490), LOOKUP (495), VIRTUAL PROXY (497), LIFECYCLE CALLBACK (501), RESOURCE POOL (503), RESOURCE CACHE (505), LAZY ACQUISITION (507), EAGER ACQUISITION (509), PARTIAL ACQUISITION (511), ACTIVATOR (513), EVICTO AUTOMATED GARBAGE COLLECTION (519), COUNTING HANDLE (522), ABSTRACT FACTORY (525), BUILDER (527), FACTORY METHOD (529), and DISPOSAL METHOD (

Database Access: DATABASE ACCESS LAYER (538), DATA MAPPER (540), ROW DATA GATEWAY (542), TABLE DATA GATEWAY (544), and ACTIVE RECORD (546).



Example of a pattern
Replicated Component Group

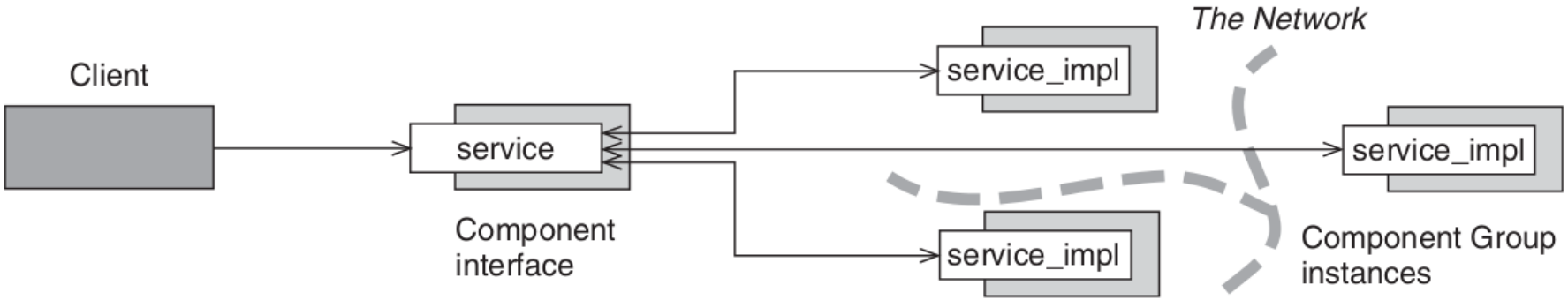
- In most systems, only a few components are exposed to extreme availability and fault tolerance requirements
  - Expensive hardware-based solutions are not justified



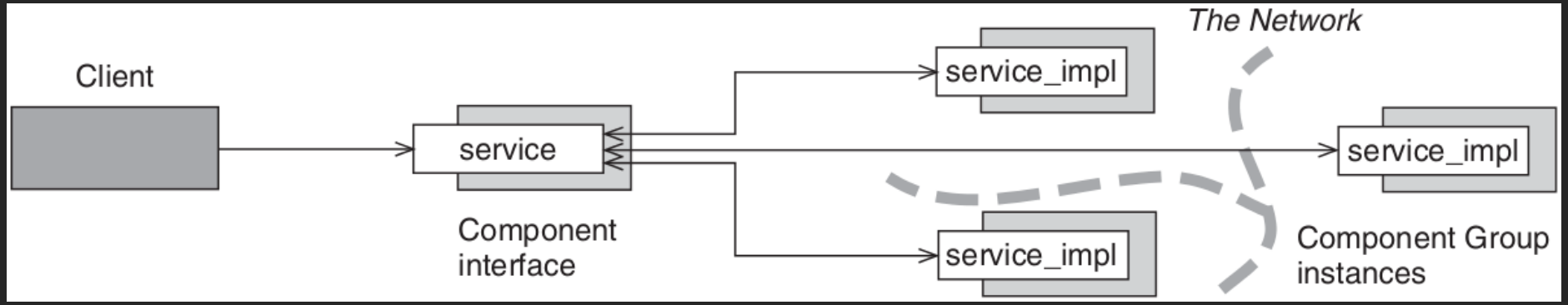
Described in Buschmann et al., Pattern-Oriented Software Architecture, Volume 4.

Replicated Component Group

- Solution
  - Provide a group of component implementations
  - Replicate these implementations across different network nodes
  - Forward client requests to all implementation instances
  - Wait until one of the instances returns a result

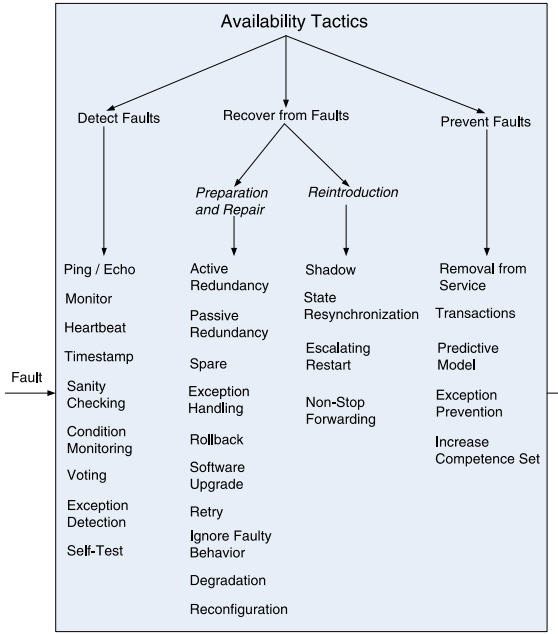


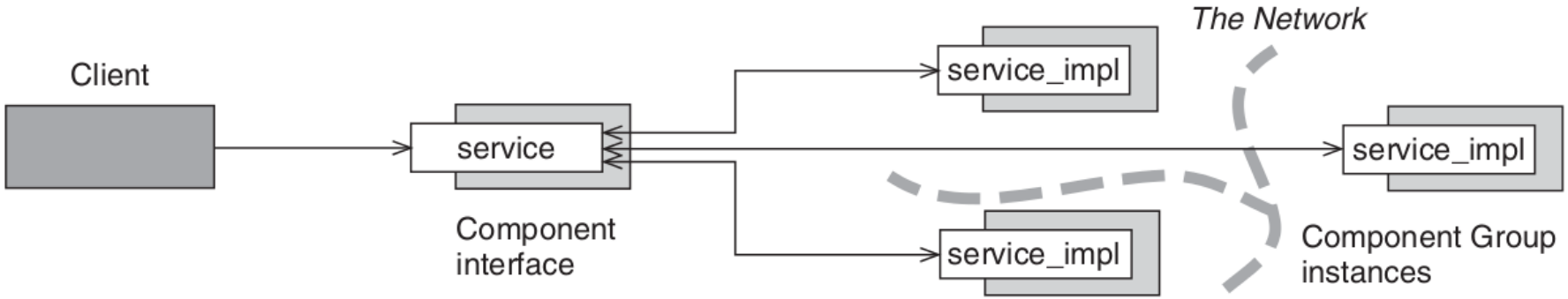
Replicated component group in course notations



Replicated Component Group

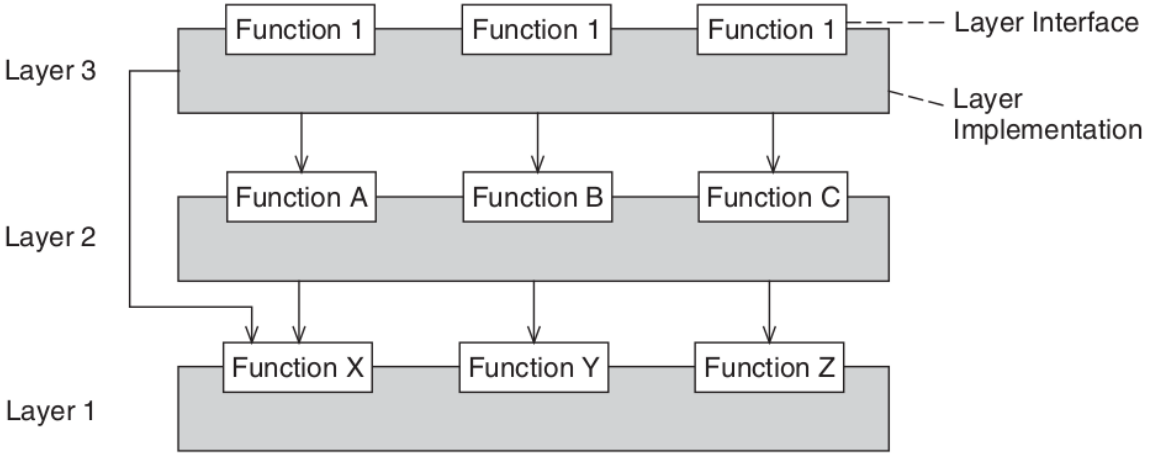
- Some tactics used by the pattern
  - Ping/Echo (for detection)
  - Active Redundancy (for recovery)





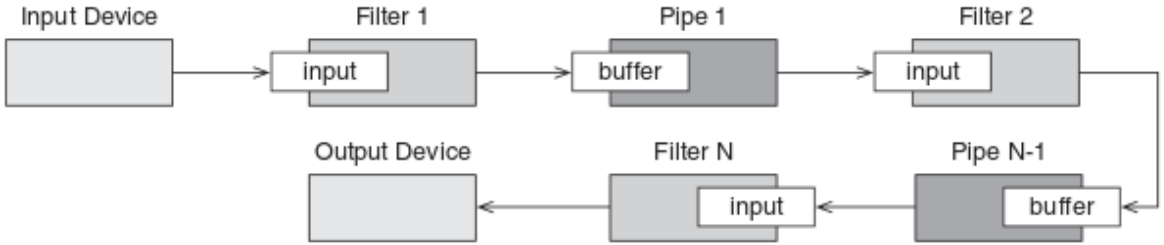
Layers pattern

- Some tactics used by the pattern
  - "Restrict dependencies"
  - "Use an intermediary"
  - "Encapsulate"



Pipe and Filter pattern (or style)

- Some tactics used by the pattern
  - "Increase semantic coherence" (filters work independently)
  - "Restrict dependencies/communication paths" (modifications are local)
  - "Encapsulate" (filters hide internal behavior)
  - "Defer binding" (pipeline configuration established at runtime)



Where do I find tactics, patterns, styles, …?

1. Your knowledge from earlier courses
2. Textbook
   - 3rd edition: chapters 5-13
3. Other resources, for example:
   - GoF Design patterns book
   - Software Architecture and Design (MSDN, online), Chapter 3: Architectural Patterns and Styles https://msdn.microsoft.com/en-us/library/ee658117.aspx
   - Frank Buschmann et al., Pattern-Oriented Software Architecture Volume 4: A Pattern Language for Distributed Computing – on Toledo
   - Software Architecture: Foundations, Theory, and Practice, Chapter 12
   - Software Systems Architecture: Working with stakeholders using viewpoints and perspectives, chapter 11.

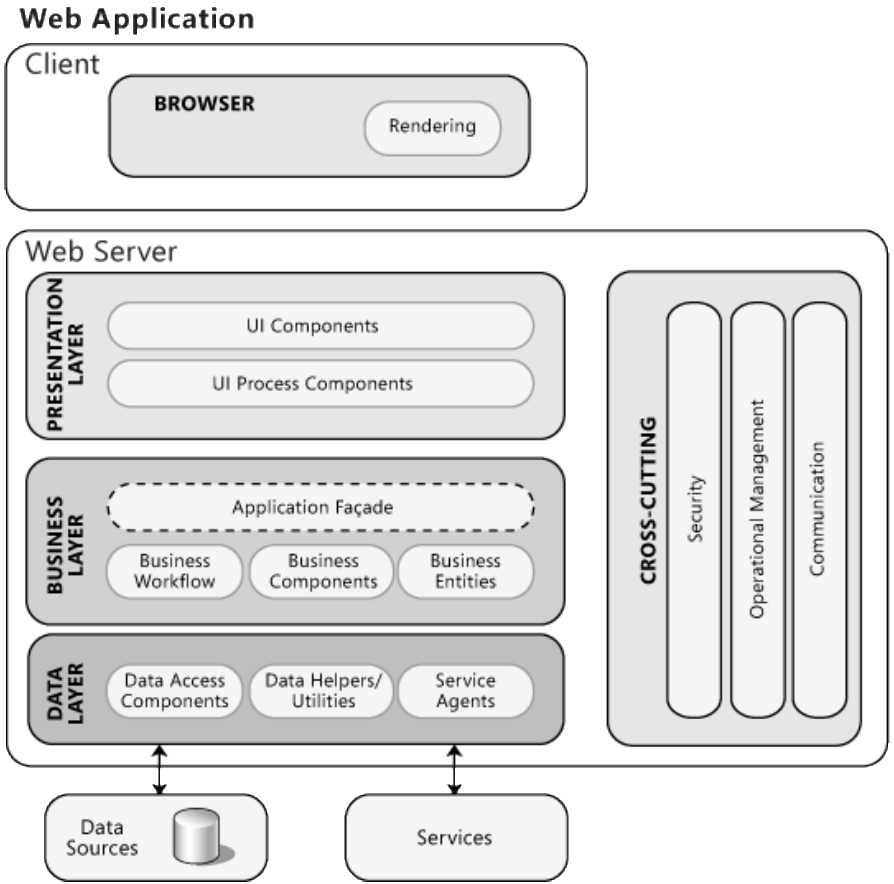
- **Keep in mind**: it doesn't have to be named 'tactic/pattern/style' to be a useful tactic/pattern/style!

4. Reference architectures

Reference architectures

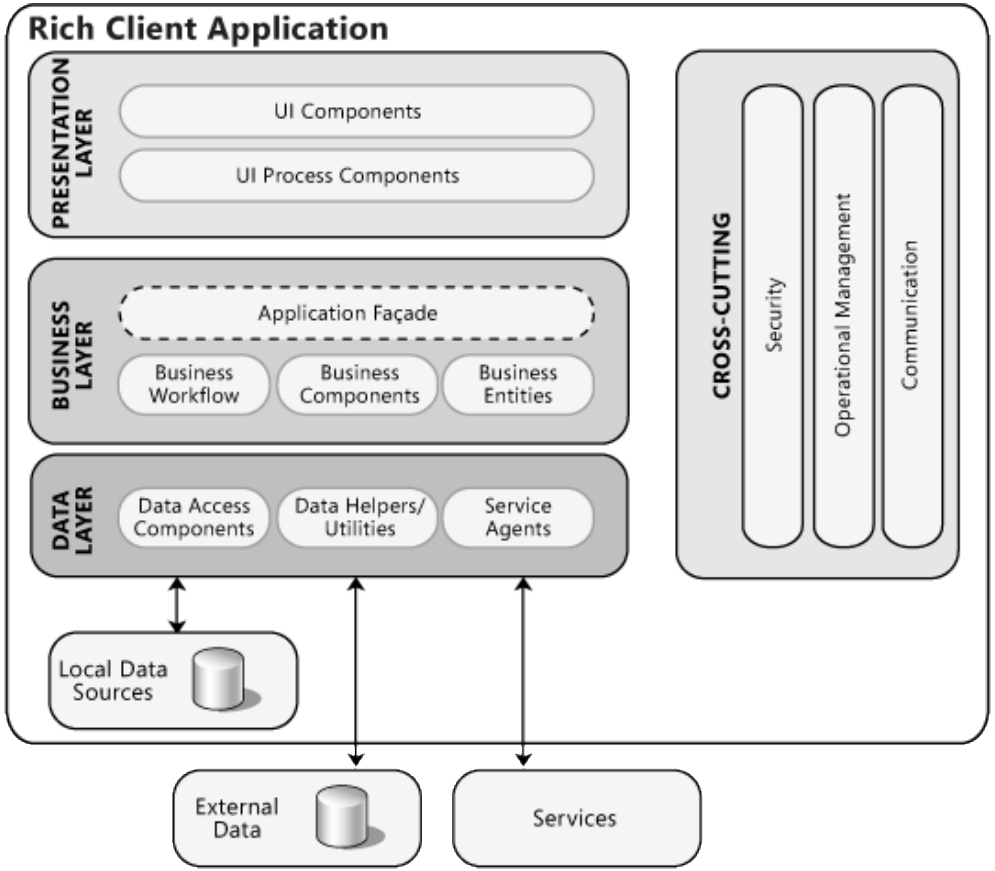
- Template **full-system scale** solution that is applicable to multiple related systems
- Sometimes within a specific application domain (ERP, CMS, …)
- Often related to standards and technology (framework)
- Concretization of certain principles, patterns and tactics

3-tiered web application



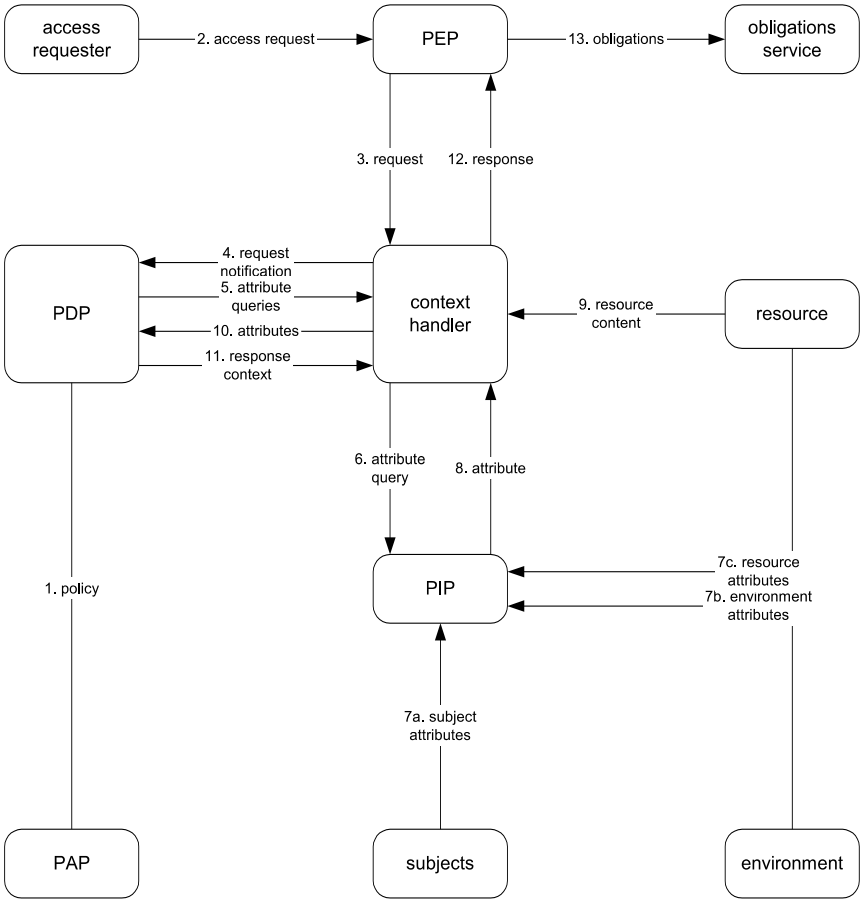
https://msdn.microsoft.com/en-us/library/ee658099.aspx

Rich client application



https://msdn.microsoft.com/en-us/library/ee658087.aspx

XACML reference architecture (access control policy)



Framework

- A framework is a reusable software element that provides generic functionality, addressing recurring concerns across a range of applications.
  - Goal: abstractions to reduce cognitive burden on programmer
- Concrete implementation of reference architectures, instantiation and ‘baked in’ tactics and patterns
- Doesn’t do anything by itself; must be used (but not modified) by a developer
- Examples: JEE, .NET, Ruby on Rails, Hibernate, Spring

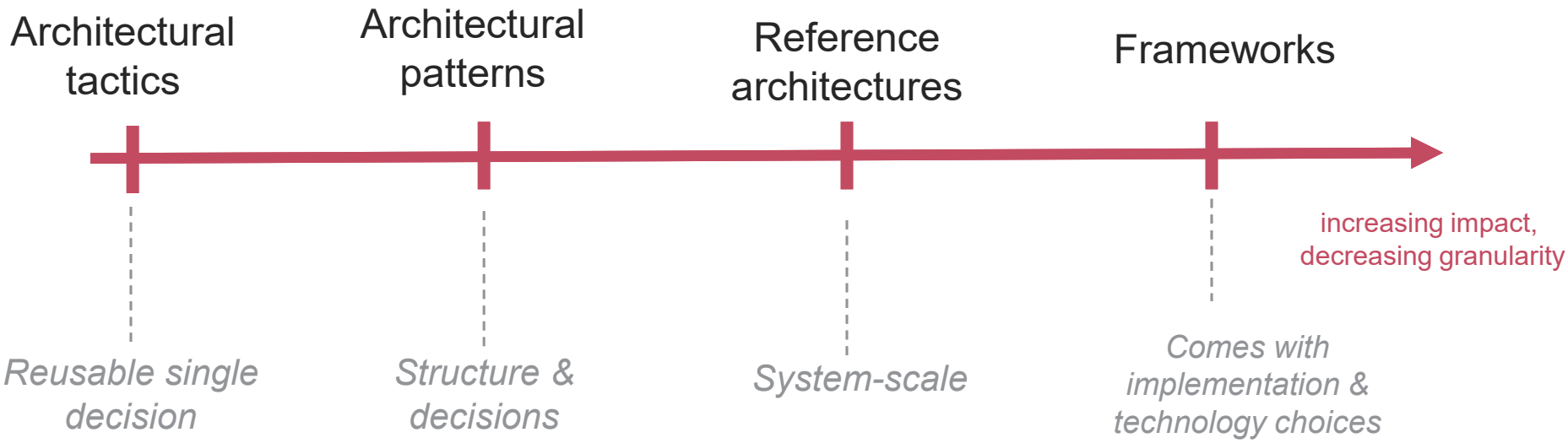
H. Cervantes, P. Velasco-Elizondo and R. Kazman, "A Principled Way to Use Frameworks in Architecture Design," in *IEEE Software*, vol. 30, no. 2, pp. 46-53, March-April 2013.

Frameworks

- By picking a framework, *many* decisions are made for you
- You must be aware of them, and ensure that they are compatible with your goals!
  - “Let’s use <*latest newest technology*> just because [it’s cool”|”everybody’s using it”] is usually a recipe for disaster

Recap

No need to reinvent the wheel



Recap

- Resources that consolidate expertise and experience of architects – *known solutions to common problems*
  1. Design principles
  2. Architectural tactics
  3. Architectural patterns
  4. Reference architectures / frameworks
- Not exhaustive – feel free to research

On Toledo:
Course Documents > Supporting materials for part 2

New definition

- **Software Architecture**
  - Making relevant (“principal”) architectural decisions at the basis of common sense, creativity, knowledge of proven solutions, and experience

    (design principles, tactics, patterns, reference architectures, ..)

Definitions of Software Architecture

- Lecture 1: "*the bridge between requirements and design*"
- Lecture 2: "*meeting or maximizing utility*"
- Lecture 3: "*{elements, form, rationale}*" | "*the set of principal design decisions*" | "*the fundamental organization of a system, embodied in its components, their relationships [..], and the principles guiding its design and evolution*"
- Lectures 4-5: a collection of *views*, [addressing *stakeholder concerns* according to well-defined *viewpoints*, further concretized in specific *models*], augmented with *rationale*
- Lecture 6: making architectural decisions at the basis of common sense, creativity, knowledge of proven solutions and experience (tactics, patterns, ..)